Inheritance

Lecture 20

Based on Slides of Dr. Norazah Yusof

Introduction to Inheritance

- ***Inheritance*** **is one of the main techniques of object-oriented programming (OOP)**
- ***Inheritance*** **models the *is-a* relationship that consists of superclass and subclass.**
- **Superclass is a general class and subclass is a specialized class.**
- **Specialized classes are said to *inherit* the methods and instance variables of the superclass.**

The "is a" Relationship

- The relationship between a superclass and an inherited class is called an "is a" relationship.
  - A grasshopper "is a" insect.
  - A poodle "is a" dog.
  - A car "is a" vehicle.
- A specialized object has:
  - all of the characteristics of the general object, plus
  - additional characteristics that make it special.
- In OOP, *inheritance* is used to create an "is a" relationship among classes.

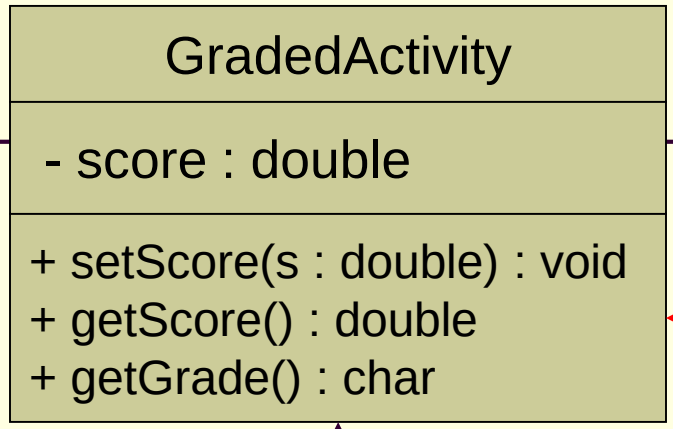
The “is a” Relationship

- We can *extend* the capabilities of a class.
- Inheritance involves a superclass and a subclass.
  - The *superclass* is the general class and
  - the *subclass* is the specialized class.
- The subclass is based on, or extended from, the superclass.
  - Superclasses are also called *base classes*, and
  - Subclasses are also called *derived classes.*
- The relationship of classes can be thought of as *parent classes* and *child classes*.

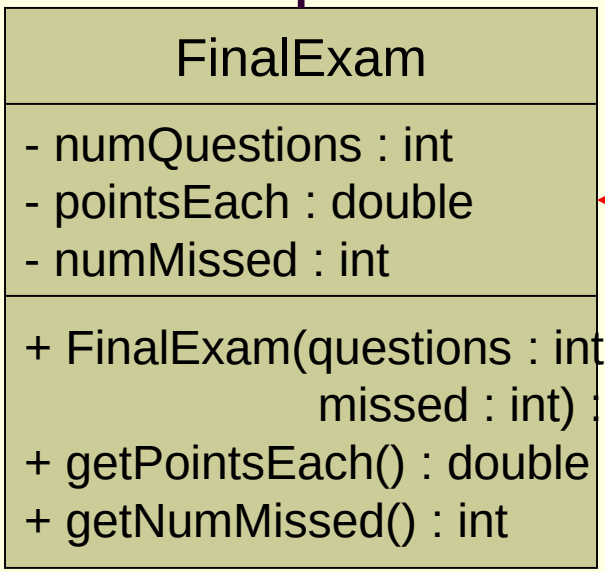
Introduction to Inheritance

- **Inheritance is the process by which a new class is created from another class**
  - **The new class is called a *derived class***
  - **The original class is called the *base class***
- **A derived class automatically has all the instance variables and methods that the base class has, and it can have additional methods and/or instance variables as well**
- **Inheritance is especially advantageous because it allows code to be *reused*, without having to copy it into the definitions of the derived classes**

The GradedActivity Example



Contains those attributes and methods that are shared by all graded activities.



Contains those attributes and methods that are specific to the FinalExam class. Inherits all non-private attributes and methods from the GradedActivity class.

- Example:
  - GradedActivity.java,
  - GradeDemo.java,
  - FinalExam.java,
  - FinalExamDemo.java

Inheritance, Fields and Methods

- **Members of the superclass that are marked *private*:**
  - **exist in memory when the object of the subclass is created, but can not be accessed by the subclass,**
  - **may only be accessed from the subclass by public accessor methods of the superclass.**
- **Members of the superclass that are marked *public*:**
  - **are inherited by the subclass, and**
  - **may be directly accessed from the subclass.**

Protected Modifier

- **Protected modifier is used to control access to the members of a class.**
- **A variable or method declared with protected visibility may be accessed by any class in the same package. In other words a derived class can reference it.**
- **A protected visibility allows a class to retain some encapsulation properties. However, the encapsulation is not as tight as private.**

Inherited Members

- **A derived class automatically has all the instance variables, static variables, and public methods of the base class**
  - **Members from the base class are said to be *inherited***
- **Definitions for the inherited variables and methods do not appear in the derived class**
  - **The code is reused without having to explicitly copy it, unless the creator of the derived class redefines one or more of the base class methods**

Parent and Child Classes

- **A base class → *parent class***
- **A derived class → *child class***
- **These relationships are often extended such that a class that is a parent of a parent . . . of another class is called an *ancestor class***
  - **If class `A` is an ancestor of class `B`, then class `B` can be called a *descendent* of class `A`**

Example Derived Classes

- When designing certain classes, there is often a natural hierarchy for grouping them
  - In a book record-keeping program, there are dictionaries, encyclopeadia and other type of books.
  - Within Java, a class called `Book` can be defined that includes all type of books.

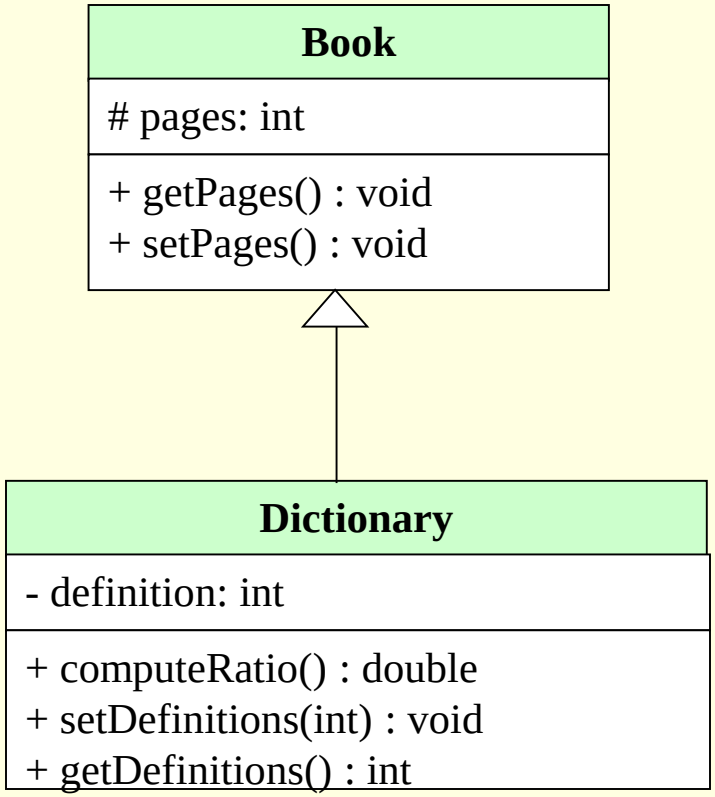
Example Derived Classes

- **Since a dictionary is a book, it is defined as a *derived* class of the class `Book`**
  - **A *derived class* is defined by adding instance variables and methods to an existing class**
  - **The existing class that the derived class is built upon is called the *base class***
  - **The phrase `extends BaseClass` must be added to the derived class definition:**

```
public class Dictionary extends Book
```

Example Derived Classes

- **In UML an arrow with an open arrowhead is used to show inheritance.**



Example Derived Classes

- Class `Book` defines the instance variables `pages` in its class definition
- Class `Dictionary` also has these instance variables, but they are not specified in its class definition.
- Class `Dictionary` has additional instance variable `definitions` that are specified in its class definition.

Example Derived Classes

- **Just as it inherits the instance variables of the class `Book`, the class `Dictionary` inherits all of its methods as well**
  - **The class `Dictionary` inherits the methods `getPages`, and `setPages` from the class `Book`**
  - **Any object of the class `Dictionary` can invoke one of these methods, just like any other method.**

Are superclass's Constructor Inherited?

No. They are not inherited.

They are invoked explicitly or implicitly.

Explicitly using the super keyword.

A constructor is used to construct an instance of a class. Unlike properties and methods, a superclass's constructors are not inherited in the subclass. They can only be invoked from the subclasses' constructors, using the keyword super. *If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.*

Inheritance and Constructors

- **Constructors are not inherited.**
- **When a subclass is instantiated, the superclass default constructor is executed first (implicitly).**
- **Example:**
  - **SuperClass1.java**
  - **SubClass1.java**
  - **ConstructorDemo1.java**

Inheritance and Constructors (Example)

```
public class SuperClass1 {
   public SuperClass1() {  /** Constructor   */
          System.out.println("This is the " +
              "superclass constructor.");
    }
}
public class SubClass1 extends SuperClass1 {
    public SubClass1()  {/** Constructor   */
        // implicit call
       System.out.println("This is the " +
                   "subclass constructor.");
    }
}
public class ConstructorDemo1 {
    public static void main(String[] args) {
       SubClass1 obj = new SubClass1();
    }
}
```

Calling The Superclass Constructor

- **If a parameterized constructor is defined in the superclass,**
  - **the superclass must provide a no-arg constructor, or**
  - **subclasses must provide a constructor, and**
  - **subclasses must call a superclass constructor.**
- **Calls to a superclass constructor must be the first java statement in the subclass constructors.**

Inheritance and Constructors (Example super keyword)

```
public class SuperClass2  {
   public SuperClass2()   { // Constructor #1
      System.out.println("This is the superclass " +
                         "no-arg constructor.");
   }
   public SuperClass2(int arg) { // Constructor #2
      System.out.println("The following argument was" +
    "passed to the superclass constructor: " + arg);
   }
}
public class SubClass2 extends SuperClass2 {
   public SubClass2() {  // Constructor
      super(10); // explicit call
      System.out.println("This is the " +
                  "subclass constructor.");
   }
}
public class ConstructorDemo2  {
   public static void main(String[] args)   {
      SubClass2 obj = new SubClass2();
   }
}
```

The difference

```
public class SuperClass1 {
   public SuperClass1() { ... } }

public class SubClass1 extends SuperClass1 {
    public SubClass1()  {// implicit call
        System.out.println("subclass constructor.");    }
   }
```

```
public class SuperClass2  {
    public SuperClass2()    { ... }
   public SuperClass2(int arg) { ... } }

public class SubClass2 extends SuperClass2 {
   public SubClass2() {  // Constructor
       super(10); // explicit call
       System.out.println("subclass constructor."); }}
```

The `super` Constructor

- **A call to the base class constructor can never use the name of the base class, but uses the keyword `super` instead**
- **A call to `super` must always be the first action taken in a constructor definition**
- **An instance variable cannot be used as an argument to `super`**

The `super` Constructor

- **If a derived class constructor does not include an invocation of `super`, then the no-argument constructor of the base class will automatically be invoked**
  - **This can result in an error if the base class has not defined a no-argument constructor**
- **Since the inherited instance variables should be initialized, and the base class constructor is designed to do that, then an explicit call to `super` should always be used**

Self Test

- What's wrong with this program?

```
public class B extends A {
    public B ( ) {
                int init = 10;
                super(40);
    }
```

Constructor Chaining

Constructing an instance of a class invokes all the superclasses' constructors along the inheritance chain. This is called *constructor chaining*.

```
public class Faculty extends Employee {
  public static void main(String[] args) {
    new Faculty();       }

  public Faculty() {
    System.out.println("(4) Faculty's no-arg constructor is invoked");
  }     }

class Employee extends Person {
  public Employee() {
    this("(2) Invoke Employee's overloaded constructor");
    System.out.println("(3) Employee's no-arg constructor is invoked");
}

  public Employee(String s) {
    System.out.println(s);
  }     }

class Person {
  public Person() {
    System.out.println("(1) Person's no-arg constructor is invoked");
  }     }
```

The keyword `this`

- A constructor may invoke an overloaded constructor or its superclass's constructor.
- The keyword `this` can be used inside a constructor to invoke another constructor of the same class (overload constructor).
- In general, a constructor with no or fewer arguments can invoke the constructor with more arguments using `this(arg-list)`.

Example on the Impact of a Superclass without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor is invoked");
  }
}
```

Example on the Impact of a Superclass without no-arg Constructor

Find out the errors in the program:

```
public class Apple extends Fruit {
}

class Fruit {
  public Fruit(String name) {
    System.out.println("Fruit's constructor is invoked");
  }
}
```

The program cannot be compiled. Since Apple is subclass of Fruit, Apple's default constructor automatically invokes Fruit's no-arg constructor. However, Fruit does not have a no-arg constructor because Fruit has an explicit constructor defined.

Example on the Impact of a Superclass without no-arg Constructor

Fixing the problem:

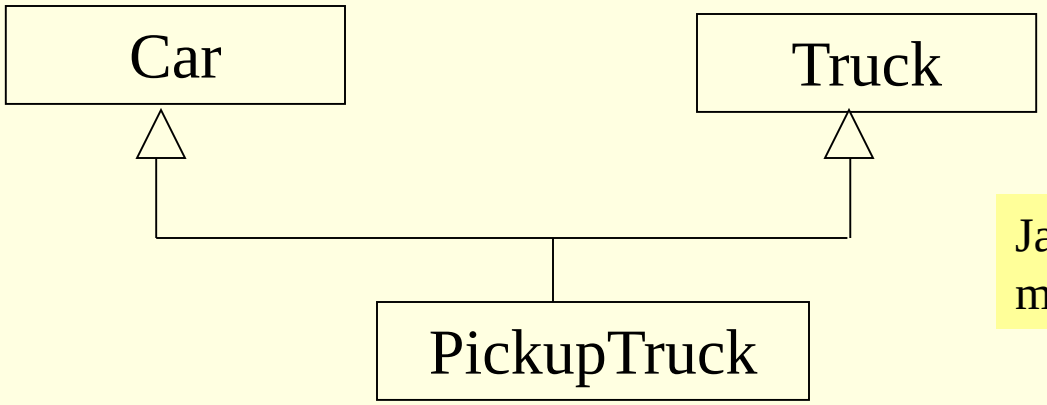
```
public class Apple extends Fruit {

}

class Fruit {
      public Fruit() {  }
      public Fruit(String name) {
    System.out.println("Fruit's constructor is
invoked");
  }
}
```

Multiple Inheritance

- **Java's approach to inheritance is called *single inheritance*, meaning a derived class can have only one parent.**
- **Some OO language allow child class to have multiple parents, i.e. *multiple inheritance*.**



Java language not to support multiple inheritance.